

Lab 08: Headless Codex Exec for CI

Course: Agentic AI Applications with Codex (C695)

Version: v1.0

Individual folder: labs/lab-08-headless-exec/

Objective

Design a non-interactive Codex run that emits machine-readable events and fails safely when acceptance checks fail.

Workflow map

Define task → Choose JSON → Run exec → Parse events → Check exit → Archive evidence

Expected output

CI-safe command wrapper and JSONL evidence

Safety boundary

Use only this lab's synthetic files. Never paste credentials or personal data into a prompt. Keep destructive, external and repository-publishing actions out of scope unless the trainer explicitly stages a safe simulation.

Folder contents

- `starter/`: learner starting state and focused tests
- `solution/`: verified reference state and expected output
- `scripts/verify_lab.py`: self-contained verifier
- `prompt.md`: bounded Codex task contract
- `evidence/checklist.md`: evidence capture template
- `AGENTS.md`: lab-local operating rules

Detailed procedure

1. Open `lab-08-headless-exec/` and read this guide and `evidence/checklist.md` before starting.
2. Open a terminal in the lab folder. Run `python3 scripts/verify_lab.py`; confirm the reference solution passes.
3. Copy `starter/` to a temporary learner working directory inside this lab. Preserve the supplied starter unchanged for comparison.
4. Read `AGENTS.md` from this folder downward. Write the applicable rule chain in the evidence checklist.
5. Give Codex the bounded task in `prompt.md`: Run the supplied repair task non-interactively. Emit JSON events, preserve `stderr`, and return non-zero if the required tests do not pass.
6. Ask Codex to inspect first and return a short plan. Check the proposed files and commands against the declared scope.

7. Approve only safe, exact actions needed for the task. Do not grant broad filesystem, network or destructive access.
8. Review every changed file in the diff. Reject unrelated edits and any embedded credential, personal data or external side effect.
9. Run the focused tests, then the supplied verifier. Record commands, exit codes and the observable result.
10. Compare the result with the acceptance rule: The wrapper avoids embedded secrets, records structured events and distinguishes success from failure by exit status.
11. Trigger the controlled failure described below, observe its evidence, then restore the accepted state.
12. Complete the evidence checklist. Keep screenshots free of secrets and submit only the artifacts requested by the trainer.

Controlled failure

Temporarily run the test against the unmodified starter. Observe the failed priority-filter criterion. Do not alter the expected test. Restore or complete the causal implementation, then re-run focused and regression checks.

Troubleshooting

Observed issue	Likely cause	Remedial action
Codex edits before explaining scope	Prompt omits inspection and finish conditions	Stop, restore the clean starter copy, and re-run with an inspect-first contract.
A command is blocked	Sandbox or approval policy is narrower than the action	Read the exact block, reduce scope, or request a precise approval from the trainer.
Tests cannot import taskboard	Test command ran from the wrong folder	Run <code>python3 -m unittest -v test_taskboard.py</code> from <code>starter/</code> or <code>solution/</code> .
Result sounds complete but evidence is absent	Finish condition did not name observable checks	Require the test output, reviewed diff and affected-file list before accepting completion.

Acceptance check

The wrapper avoids embedded secrets, records structured events and distinguishes success from failure by exit status.

A plausible answer is not sufficient. The pass decision requires the observed changed state, test output, reviewed diff and named human decision where the workflow crosses a consequential boundary.